

ISM2411 — Lab Week 12

Read a Sales CSV, Write a Cleaned Report — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 12 · Unit 3 · Data Structures

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Segments	3
6 Segment-by-Segment Walkthrough	4
6.1 Intro (0:00–0:04)	4
6.2 Exercise 1 — Get the File (0:04–0:08, 4 min)	4
6.3 Exercise 2 — Read and Print Raw Rows (0:08–0:16, 8 min)	6
6.4 Exercise 3 — Skip the Header (0:16–0:22, 6 min)	8
6.5 Exercise 4 — Compute Totals (0:22–0:31, 9 min)	10
6.6 Exercise 5 — Filter and Write (0:31–0:41, 10 min)	12
6.7 Exercise 6 — <code>csv.DictReader</code> (0:41–0:48, 7 min)	15
6.8 Exercise 7 — Sales Summary by Product (0:48–0:56, 8 min)	17
6.9 Exercise 8 — Append Mode Log (0:56–1:06, 10 min)	19
7 Wrap-Up (last ~9 minutes)	21
8 Appendix A — Full Answer Key (<code>csv_practice.py</code>)	21
9 Appendix B — Sample <code>sales.csv</code> (verified)	24
10 Appendix C — Extra Practice (only if the class finishes early)	24

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 12 Lab — Read a Sales CSV, Write a Cleaned Report
Unit	Unit 3 · Data Structures
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 11: dictionaries, <code>.get()</code> accumulate-into-a-dict pattern; Module 10: lists
Student-facing lab page	markumreed.github.io/ism2411 — week12_lab
Exercises covered	Exercises 1–8 (required)
Submission	<code>csv_practice.py</code> + the cleaned CSV and other output files, via GitHub (week12/ folder), repo URL to Canvas

Every prior module's data lived only inside the running script — created fresh each run, gone the moment the program ended. This is the first lab where a program's *inputs and outputs are real files on disk*, and the lab page's own framing is worth repeating to the class verbatim: this is a first complete **ETL pipeline** (Extract, Transform, Load) — read raw data in, transform it (filter, compute, summarize), and write cleaned results back out. This exact shape — read, transform, write — is the skeleton of an enormous fraction of real business data work. Protect real time for the `with` block (Exercise 2) and the `newline=""` detail (Exercise 5) — both are easy to wave past but cause genuinely confusing bugs if skipped.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Open a file safely with a `with` block, and explain why this is preferred over manually calling `open()/close()`.
2. Read a CSV file with `csv.reader`, understand that every value comes back as a string, and use `next()` to separate a header row from data rows.
3. Read the same file more robustly with `csv.DictReader`, accessing fields by column name instead of position, and articulate when each approach is preferable.
4. Write a new CSV file with `csv.writer`, including the `newline=""` detail required on some platforms to avoid blank rows.
5. Combine Module 11's accumulate-into-a-dictionary pattern with file reading to build a summary report from raw data.

6. Append to a text log file across multiple separate script runs, using `datetime` to timestamp each entry.

3 Before Class — Setup Checklist

- ☐ Confirm `sales.csv` is available on Canvas for distribution, or prepare a 10-row sample yourself — Appendix B has a verified sample file with realistic mixed order sizes (deliberately including a few orders under \$100, so Exercise 5's filter has something visible to exclude).
- ☐ Create the `week12/data/` folder structure on your own demo machine before class, and confirm your own script runs end to end once, top to bottom, before live-coding it in front of the room.
- ☐ Decide how you'll demonstrate Exercise 8's "run the script three times" requirement live — e.g., running the same script three times from the terminal between short explanatory pauses — since seeing the log file grow, not just being told it will, is the actual point of that exercise.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+ (the `csv` and `datetime` modules are both part of the standard library — no `pip install` needed)
- Students: `sales.csv` (provided or self-created), same GitHub repo with a new `week12/data/` folder

5 Segments

The required exercises fill the available time.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Today your program reads a real file from disk, transforms the data inside it, and writes a new file back out — a complete pipeline: Extract, Transform, Load, or ETL for short. This exact pattern — read raw data in, clean or summarize it, save the result — is one of the most common shapes of real business data work, in any language, at any company.”

Do: Open `csv_practice.py`, type the header:

```
# csv_practice.py
# ISM2411 Module 12 Lab – Read a Sales CSV, Write a Cleaned Report
```

6.2 Exercise 1 — Get the File (0:04–0:08, 4 min)

Teaching goal: Confirm the input file exists at the correct relative path before writing any code that depends on it — a genuinely important habit, since “file not found” is the single most common failure mode in this entire lab.

Say to the class:

“Before any Python, confirm the file itself is where we expect it. `sales.csv` needs to live at `week12/data/sales.csv`, relative to wherever your script runs from — get this path wrong and every exercise from here fails with the same unhelpful-looking error.”

Do: Have every student create the folder structure and place the file:

```
mkdir -p week12/data
# place sales.csv inside week12/data/
```

Verify with a minimal script:

```
with open("week12/data/sales.csv") as f:
    print("File opened successfully")
```

Line-by-line explanation:

- The path `"week12/data/sales.csv"` is a **relative path**, exactly like Module 01’s directory-tree exercises — it’s relative to wherever the script itself is run *from*, not relative to the script file’s own location. This distinction genuinely matters here: a student running the script from inside `week12/` versus from the repo root will need a different relative path, and this is worth stating explicitly as the single most common setup issue in this lab.

Run it. Expected output:

File opened successfully

Common student mistakes to watch for:

- Running the script from the wrong directory (a direct callback to Module 02 Exercise 7) — produces `FileNotFoundError: [Errno 2] No such file or directory: 'week12/data/sales.csv'`. Have students run `pwd` first and reason about the relative path from there, rather than guessing and re-running repeatedly.
- Placing the file at `week12/sales.csv` (missing the `data/` subfolder) or naming it something slightly different (`Sales.csv`, case mismatch on a case-sensitive filesystem) — both produce the identical `FileNotFoundError`, worth explicitly checking both possibilities if the error persists after confirming the working directory is correct.

Check for understanding: “If I moved `sales.csv` to sit directly next to my script instead of inside `week12/data/`, what — if anything — would I need to change in my code?” (The path string itself, from `"week12/data/sales.csv"` to just `"sales.csv"` — get a student to state explicitly that the code has to match wherever the file actually lives; there’s no automatic discovery.)

6.3 Exercise 2 — Read and Print Raw Rows (0:08–0:16, 8 min)

Teaching goal: Open a file safely with a `with` block, read it with `csv.reader`, and observe — critically — that every value comes back as a **string**, even ones that look numeric.

Say to the class:

“New syntax today: `with`. This is how you open a file safely — it guarantees the file gets closed automatically, even if something goes wrong partway through, without you having to remember to call `.close()` yourself.”

Live-code this:

```
# --- Exercise 2 ---
import csv

with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Line-by-line explanation:

- `import csv` — the `csv` module is part of Python’s standard library (no installation needed) and provides tools specifically for reading/writing this common file format correctly, including edge cases like commas *inside* quoted fields, which a naive manual `.split(",")` approach would get wrong.
- `with open("week12/data/sales.csv") as f:` — **this is the new syntax to slow down for.** `with` creates a block where `f` (the open file) is available, and — crucially — automatically closes the file the moment the block ends, even if an error occurs partway through. Contrast this explicitly with the older, error-prone pattern `f = open(...); ...; f.close()`, where a forgotten or skipped `.close()` (e.g., because an error occurred before reaching it) leaves the file open, potentially causing resource leaks or file-locking issues on longer-running programs. Say plainly: **with is the standard, correct way to open a file in modern Python — there’s essentially no reason to use the manual open/close pattern instead.**
- `reader = csv.reader(f)` — wraps the open file in a CSV reader object, which knows how to correctly split each line into a list of field values, respecting CSV formatting rules.
- `for row in reader:` — each row is a **list of strings**, one per pass through the loop, including the header row on the very first iteration — this loop doesn’t yet distinguish header from data, which is exactly what Exercise 3 fixes.

Run it. Expected output (first three of eleven lines shown; header included):

```
['product', 'price', 'quantity']
['Widget A', '9.99', '50']
```

```
['Widget B', '24.99', '12']  
...
```

Point out explicitly, since it's the exercise's stated goal: every value — even '9.99' and '50' — is a **string**, wrapped in quotes in the printed output. Nothing has been converted to a number yet; that's a deliberate, separate step coming in Exercise 4, exactly parallel to `input()` always returning strings back in Module 03.

Common student mistakes to watch for:

- Forgetting `import csv` at the top of the file — raises `NameError: name 'csv' is not defined` the moment `csv.reader` is referenced; a good, familiar error type from many prior modules, now in a new context (a missing import, not a typo).
- Trying to do math directly on a row's values without conversion (`row[1] * row[2]`) — since both are strings, this either raises `TypeError` or does something unexpected depending on the specific values, directly foreshadowing Exercise 4's required `float()/int()` conversions.

Check for understanding: “How many total rows print, including the header — and how do you know without counting the printed lines by hand?” (Eleven — ten data rows plus one header row; a student who opened the CSV directly in a text editor beforehand and counted lines there has a good independent way to confirm the loop visited every row.)

6.4 Exercise 3 — Skip the Header (0:16–0:22, 6 min)

Teaching goal: `next()` to advance an iterator by exactly one step, pulling the header out of the loop entirely — a small piece of syntax with an outsized payoff for every remaining exercise today.

Say to the class:

“One new function, `next()`, and it solves the ‘header row keeps getting mixed in with data’ problem for the rest of this lab.”

Live-code this:

```
# --- Exercise 3 ---
with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    print(header)
    for row in reader:
        print(row)
```

Line-by-line explanation:

- `header = next(reader)` — `reader` is what’s called an **iterator**: something that hands out one item at a time when asked. `next(reader)` explicitly asks for “the next item” — here, called *before* the loop even starts, so it pulls exactly the *first* row (the header) out and stores it in `header`, **advancing the reader’s internal position by one**. This is the key mechanism to state explicitly: after this line runs, the reader “remembers” it already gave out the header, so the `for` loop that follows starts fresh from the *second* row onward — the header is never seen by the loop at all.
- `print(header)` — printed once, separately, confirming it was captured correctly and distinctly from the data rows.
- `for row in reader:` — now starts at the first genuine data row, since `next(reader)` already consumed the header before the loop began.

Run it. Expected output:

```
['product', 'price', 'quantity']
['Widget A', '9.99', '50']
['Widget B', '24.99', '12']
...
```

(Ten data rows follow the header line, with no repeated header this time.)

Common student mistakes to watch for:

- Calling `next(reader)` *inside* the loop, expecting it to skip the header on the first iteration — this actually skips a *different* row each time depending on where it’s placed, and is a much

more confusing, error-prone pattern than calling it once, cleanly, before the loop starts. If a student tries this, walk through what it actually does with them rather than just saying “put it before the loop instead” — understanding *why* the placement matters is the real lesson.

- Assuming `next()` can be called again later to “reset” back to the header — it can’t; a `csv.reader` only moves forward, never backward. If a script needs the header again later, storing it in a variable (as `header = next(reader)` does) is the correct approach, not calling `next()` a second time.

Check for understanding: “If I called `next(reader)` a *second* time, right after the first, before starting the loop, what would `header` end up holding, and what would the loop start on?” (The second call would actually consume and discard the *first data row* instead of the header, storing that row’s content in `header` if reassigned, or simply skipping it if not — and the loop would then start from the *second* data row. A good trace-it-by-hand question confirming students understand `next()` moves forward one step at a time, unconditionally, regardless of what’s “supposed” to be there conceptually.)

6.5 Exercise 4 — Compute Totals (0:22–0:31, 9 min)

Teaching goal: Convert string values to numbers, compute a per-row total, and accumulate an overall total — combining Module 03’s type-conversion lesson with Module 06’s accumulator pattern, now reading from a real file instead of a hardcoded list.

Say to the class:

“Same conversion requirement as `input()` back in Module 3 — everything from this CSV is a string, and we need real numbers to compute revenue. Then the exact accumulator pattern from Module 6, totaling revenue across every row.”

Live-code this:

```
# --- Exercise 4 ---
total_revenue = 0                # initialize
with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    for row in reader:
        product, price, quantity = row
        price = float(price)
        quantity = int(quantity)
        line_total = price * quantity
        total_revenue += line_total # update
        print(f"{product}: {quantity} units × ${price:.2f} = ${line_total:.2f}")

print(f"Total revenue: ${total_revenue:.2f}") # use
```

Line-by-line explanation:

- `total_revenue = 0` — the accumulator, initialized **before** the `with` block even opens — say explicitly, echoing Module 06: this has to exist before the loop starts, for the same reason as always.
- `product, price, quantity = row` — **tuple unpacking**, from Module 01, applied to a CSV row for the first time: since every row is a three-element list (`['Widget A', '9.99', '50']`), this one line assigns all three fields to meaningfully-named variables at once, instead of writing `row[0]`, `row[1]`, `row[2]` throughout the rest of the loop body.
- `price = float(price)` and `quantity = int(quantity)` — the required conversions, exactly parallel to Module 03/06’s `input()` conversions — say explicitly: **a CSV’s values need type conversion for the same underlying reason `input()`’s values do — both hand back plain text, regardless of what the text looks like.**
- `total_revenue += line_total` — the familiar accumulator update, now inside a loop reading from a file rather than iterating a hardcoded list — worth pointing out explicitly that the

accumulator pattern itself hasn't changed at all; only the *source* of the loop's values has.

Run it. Expected output (verified against the sample `sales.csv` in Appendix B):

```
Widget A: 50 units × $9.99 = $499.50
Widget B: 12 units × $24.99 = $299.88
Gadget C: 3 units × $49.99 = $149.97
Gadget D: 20 units × $14.99 = $299.80
Gizmo E: 2 units × $99.99 = $199.98
Doohickey F: 80 units × $4.99 = $399.20
Widget A: 5 units × $9.99 = $49.95
Gadget C: 1 units × $49.99 = $49.99
Widget B: 2 units × $24.99 = $49.98
Gizmo E: 1 units × $99.99 = $99.99
Total revenue: $2098.24
```

Common student mistakes to watch for:

- Forgetting either conversion (leaving price or quantity as a string) — price * quantity between a string and an int actually *runs without error* if only quantity is converted (string-times-int means repetition, from Module 03's Exercise 5 extra) producing a bizarre repeated-text result rather than a clean crash; this is worth demonstrating live specifically because it's a *silent*, not obviously broken, failure mode, distinct from the cleaner `TypeError` that occurs if *neither* value is converted.
- Reusing `total` as a variable name (shadowing Module 06's habitual name) without noticing it collides with nothing here — harmless in isolation, but worth a brief naming-hygiene note now that files this large are accumulating many exercises' worth of variables in one script.

Check for understanding: “If `sales.csv` had 500 rows instead of 10, what — if anything — needs to change about this code to still work correctly?” (Nothing — the `with/csv.reader/for` combination processes however many rows exist automatically, exactly like Module 06's loops didn't care how long `sales` was. This is worth stating explicitly as the real payoff of reading from a file with a loop, rather than, say, manually typing ten separate conversions.)

6.6 Exercise 5 — Filter and Write (0:31–0:41, 10 min)

Teaching goal: Write a *new* CSV file, containing only rows that pass a filter — combining `csv.writer` with the filtering pattern from Module 10 Exercise 8, and encountering the `newline=""` detail that prevents a genuinely common, confusing bug.

Say to the class:

“Now we write, not just read — a brand-new CSV, containing only the orders over \$100. One easy-to-miss detail in the `open()` call that avoids a real, confusing bug: `newline=""`. I’ll show you what goes wrong without it.”

Live-code this:

```
# --- Exercise 5 ---
rows_written = 0
rows_total = 0

with open("week12/data/sales.csv") as infile, \
    open("week12/data/cleaned.csv", "w", newline="") as outfile:
    reader = csv.reader(infile)
    writer = csv.writer(outfile)
    header = next(reader)
    writer.writerow(["Product", "Price", "Qty", "Total"])

    for row in reader:
        rows_total += 1
        product, price, quantity = row
        price = float(price)
        quantity = int(quantity)
        total = price * quantity
        if total > 100:
            writer.writerow([product, price, quantity, round(total, 2)])
            rows_written += 1

print(f"Wrote {rows_written} of {rows_total} rows")
```

Line-by-line explanation:

- `with open(...)` as `infile`, `open(...)` as `outfile`: — **two files open at once**, in a single `with` statement — say explicitly this is valid, common syntax: separate them with a comma, and both get automatically closed at the end of the block, regardless of which one a student is more used to seeing alone.
- `open("week12/data/cleaned.csv", "w", newline="")` — three arguments this time, not

one: the filename, "w" (write mode — say explicitly this **creates the file if it doesn't exist, and completely overwrites it if it does** — a real, worth-flagging risk if a student accidentally points this at a file they meant to keep), and `newline=""`. **This third argument is the detail to explain carefully:** on Windows specifically, Python's default text-file handling can introduce extra blank lines between every row when combined with the `csv` module's own line-ending handling, because both layers try to manage line endings independently. Passing `newline=""` tells Python's file layer to step back and let the `csv` module handle line endings entirely on its own, avoiding the conflict. Say explicitly: **this is genuinely one of those “just always include this, it's a known gotcha” details** — not something students need to derive from first principles, just remember to include whenever writing CSVs.

- `writer = csv.writer(outfile)` — parallel structure to `csv.reader`, but for writing.
- `writer.writerow(["Product", "Price", "Qty", "Total"])` — writes the header row for the *new* file, with different column names than the original (Price capitalized, Qty abbreviated) — worth noting this is a deliberate design choice in the exercise, not something students need to preserve from the source file's naming.
- `if total > 100: writer.writerow(...)` — the actual filter, inside the loop, exactly parallel to Module 10 Exercise 8's `if sale > 300: high_sales.append(sale)` — the same “build something new, conditionally, inside a loop” shape, just writing to a file instead of appending to a list.

Run it. Expected output (verified against the sample `sales.csv`, where 4 of 10 rows fall at or below \$100):

Wrote 6 of 10 rows

And week12/data/cleaned.csv should now contain:

```
Product,Price,Qty>Total
Widget A,9.99,50,499.5
Widget B,24.99,12,299.88
Gadget C,49.99,3,149.97
Gadget D,14.99,20,299.8
Gizmo E,99.99,2,199.98
Doohickey F,4.99,80,399.2
```

Common student mistakes to watch for:

- Omitting `newline=""` — **demonstrate this live if your demo platform shows the effect** (most visible on Windows; Mac/Linux often don't show the bug at all, which is itself worth mentioning, since a student on a Mac might reasonably wonder why this matters if their own file looks fine — the honest answer is that it's a cross-platform safety habit, not something every single student will necessarily witness break on their own machine).
- Opening the output file in "w" mode by accident when they meant to *add* to an existing file rather than replace it — a good moment to preview Exercise 8's "a" (append) mode as the alternative, and to flag explicitly that "w" mode's overwrite-without-warning behavior is worth

real caution.

- Writing the filtered row using the *original* price/quantity variables from the loop but forgetting that `total` needs to be freshly computed and passed along too, rather than assumed to already exist in the new file's structure — a good moment to trace through exactly which four values `writer.writerow(...)` needs, matching the four header columns one-to-one.

Check for understanding: “If I ran this exact script a second time without changing anything, what would `cleaned.csv` contain afterward — the same six rows, twelve rows, or something else?” (Still exactly six rows — “w” mode overwrites the file completely on each run, rather than adding to what’s already there; this is the direct, important contrast with Exercise 8’s append mode coming up next.)

6.7 Exercise 6 — `csv.DictReader` (0:41–0:48, 7 min)

Teaching goal: Re-read the same file using `csv.DictReader`, accessing fields by column *name* instead of position — directly parallel to Module 11's dictionaries, and a more robust, readable alternative to `csv.reader`'s positional tuple-unpacking.

Say to the class:

“Same file, same total, different reading style — one that gives you Module 11's dictionary-style `row['product']` access instead of position-based unpacking.”

Live-code this:

```
# --- Exercise 6 ---
total_revenue = 0
with open("week12/data/sales.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        price = float(row["price"])
        quantity = int(row["quantity"])
        total_revenue += price * quantity

print(f"Total revenue (DictReader): ${total_revenue:.2f}")

# .get() vs bracket notation: DictReader rows behave like Module 11's
# dictionaries — .get() with a default is safer if a column might be
# missing from some rows; bracket notation is fine (and more explicit)
# when you're confident every row has the expected columns, as here.
```

Line-by-line explanation:

- `reader = csv.DictReader(f)` — the key difference from Exercise 2's `csv.reader`: `DictReader` **automatically uses the first row as column names**, and hands back each subsequent row as a *dictionary*, not a plain list — say explicitly: **there's no separate `next(reader)` call needed here to skip the header** — `DictReader` consumes it automatically to build the field names, and the loop that follows starts directly on the first data row.
- `row["price"]`, `row["quantity"]` — Module 11's exact dictionary-access syntax, now on data read straight from a file. Worth stating the direct payoff over Exercise 4's positional unpacking: **if the CSV's column order ever changed, this code would still work correctly**, since it looks fields up by name, not position — `product`, `price`, `quantity` = `row` from Exercise 4, by contrast, would silently assign the wrong values to the wrong variable names if the columns were ever reordered in the source file.
- The required comparison comment — this is the exercise's actual required deliverable, not

just matching totals: a good answer names DictReader's resilience to column reordering and readability (`row["price"]` reads more clearly than `row[1]`) as reasons to prefer it, and `csv.reader`'s simplicity/slightly better performance on very large files as reasons one might still choose it.

Run it. Expected output (must match Exercise 4's total exactly):

Total revenue (DictReader): \$2098.24

Common student mistakes to watch for:

- Calling `next(reader)` before the loop anyway, out of habit from Exercise 3 — this actually skips the *first real data row* by mistake, since DictReader already handled the header internally; a good moment to have the room predict the (now-wrong, missing-a-row) total this produces and connect it back to why the header-handling difference matters.
- Using the *original* column names from the source `sales.csv` (`product`, `price`, `quantity`, all lowercase) when they actually meant to reference Exercise 5's *output* file's different column names (`Product`, `Price`, capitalized) — a good reminder that DictReader's keys come directly from whatever header row is actually in the file being read, not from any exercise's "standard" naming.

Check for understanding: "Both this exercise and Exercise 4 compute the exact same total — what does that consistency actually confirm?" (That both reading approaches are correctly processing the same underlying data, just through different-looking code — a good moment to reinforce that `csv.reader` and `csv.DictReader` are two *interfaces* onto the same file, not two different computations.)

6.8 Exercise 7 — Sales Summary by Product (0:48–0:56, 8 min)

Teaching goal: Combine Module 11’s accumulate-into-a-dictionary pattern with this module’s file-reading skills — genuinely the payoff exercise that shows why both modules’ techniques matter together.

Say to the class:

“This is where Module 11 and today combine directly: build a summary dictionary — one total per product — by reading straight from the file, using the exact `.get(key, 0)` accumulator pattern from two weeks ago.”

Live-code this:

```
# --- Exercise 7 ---
summary = {}
with open("week12/data/sales.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        product = row["product"]
        total = float(row["price"]) * int(row["quantity"])
        summary[product] = summary.get(product, 0) + total

for product in sorted(summary):
    print(f"{product}: ${summary[product]:.2f}")
```

Line-by-line explanation:

- Every line here is either Module 11 Exercise 8’s dictionary-accumulator pattern (`summary = {}`, `summary.get(product, 0) + total`) or this module’s file-reading pattern (`csv.DictReader`, `row["product"]`) — say explicitly: **there’s nothing new in this exercise syntactically; it’s entirely about recognizing that two previously-separate patterns compose cleanly together.** This is worth stating as its own lesson: real programs are usually built by combining a handful of patterns you already know, not by learning an entirely new technique for every new task.
- `summary[product] = summary.get(product, 0) + total` — identical logic to Module 11 Exercise 8’s region-summary line, with `product` in place of `region` and a per-row computed `total` in place of a pre-existing `t["amount"]` field.

Run it. Expected output (sorted alphabetically by product name; verified against the sample `sales.csv`):

```
Doohickey F: $399.20
Gadget C: $199.96
Gadget D: $299.80
```

Gizmo E: \$299.97

Widget A: \$549.45

Widget B: \$349.86

Common student mistakes to watch for:

- Using `+=` directly (`summary[product] += total`) instead of the `.get()`-protected version — fails with `KeyError` on a product's first appearance, exactly the same failure mode flagged in Module 11 Exercise 8; if a student has forgotten this lesson in the two weeks since, this is a good moment for a quick, honest callback rather than re-deriving it from scratch.
- Confirming the summary total is correct by manually re-adding the source numbers — a genuinely good habit worth encouraging explicitly: e.g., Widget A appears in two rows (50 units and 5 units, both at \$9.99), and $499.50 + 49.95 = 549.45$ matches the summary exactly — a real, concrete way to build trust in the accumulator logic beyond just “the code ran without error.”

Check for understanding: “How would you modify this to also track *how many rows* contributed to each product's total, not just the revenue sum?” (Add a second dictionary — or a nested structure — tracking counts the same way, e.g. `counts[product] = counts.get(product, 0) + 1` alongside the existing revenue accumulator; getting a student to propose this extension confirms they understand the *pattern*, not just this one specific instance of it.)

6.9 Exercise 8 — Append Mode Log (0:56–1:06, 10 min)

Teaching goal: Append mode ("a") versus write mode ("w") — and datetime for real timestamps — producing a log file that accumulates history across multiple separate runs of the same script, rather than being overwritten each time.

Say to the class:

“Every file write so far has used 'w' mode — overwrite completely, every time. Now: append mode, 'a' — add to the end, keeping everything that was already there. We’re building a run log: every time this script executes, one new line gets added, with a timestamp and the computed total. I’m going to run this three separate times and we’ll watch the log grow.”

Live-code this:

```
# --- Exercise 8 ---
from datetime import datetime

def compute_total_revenue():
    total = 0
    with open("week12/data/sales.csv") as f:
        reader = csv.DictReader(f)
        for row in reader:
            total += float(row["price"]) * int(row["quantity"])
    return total

total_revenue = compute_total_revenue()
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

with open("week12/data/run_log.txt", "a") as f:
    f.write(f"{timestamp} | Total revenue: ${total_revenue:.2f}\n")
```

Line-by-line explanation:

- `from datetime import datetime` — a new import style: this pulls in the `datetime` class directly from the `datetime` module (confusingly, they share the same name) — say explicitly, since it’s a very common point of confusion: `import datetime` alone would require writing `datetime.datetime.now()` (module, then class); `from datetime import datetime` lets you write just `datetime.now()`. Both are valid Python, but this lab uses the second, shorter form.
- `def compute_total_revenue():` — Exercise 7’s total-computing logic, wrapped in a function (Module 07 skills) specifically so it can be called cleanly, once, each time the script runs, without duplicating the read-and-accumulate code inline.
- `datetime.now().strftime("%Y-%m-%d %H:%M:%S")` — `datetime.now()` captures the current date and time; `.strftime(...)` formats it into a specific, readable string layout —

say explicitly, briefly: the format codes (%Y four-digit year, %m month, %d day, %H:%M:%S hours:minutes:seconds) are a mini-language of their own, not something to memorize deeply today, just recognize as “a way to control exactly how a date/time prints.”

- `open("week12/data/run_log.txt", "a")` — **"a" for append**, not "w" — this is the single detail the entire exercise is built around. Contrast explicitly, one more time, with Exercise 5: "w" mode would **erase the log's previous contents every single run**, defeating the entire purpose of a running log; "a" mode adds a new line at the end, preserving everything written by every prior run.
- `f.write(f"...\\n")` — note this uses `.write()`, not `print()` — `.write()` writes exactly the string given, with no automatic newline added, which is why the f-string explicitly ends with `\\n` (a newline character) — omit this and every run's log entry would run together onto one single, ever-growing line instead of one line per run.

Run the script three separate times, live, from the terminal, narrating between runs. Expected run_log.txt after three runs (timestamps will differ per actual run time):

```
2024-11-04 10:32:16 | Total revenue: $2098.24
2024-11-04 10:32:19 | Total revenue: $2098.24
2024-11-04 10:32:23 | Total revenue: $2098.24
```

Point out explicitly: three separate lines, three separate timestamps, same total revenue each time (since `sales.csv` itself didn't change between runs) — this is exactly the expected, correct behavior, and a good moment to ask the room what a *changing* total across runs would imply (that the underlying `sales.csv` was modified between script executions — a genuinely useful diagnostic signal a real operations log like this would provide).

Common student mistakes to watch for:

- Using "w" instead of "a" — the single most consequential mistake in this exercise; running the script three times would leave `run_log.txt` with only the *most recent* run's single line, silently destroying the history the exercise explicitly asks students to build and observe. If this happens, have the student run it three times, look at the file, and diagnose why there's only one line themselves, rather than immediately naming the fix.
- Forgetting the trailing `\\n` in the write string — all three runs' text ends up concatenated onto one visually confusing single line in the file, technically present but hard to read; a good moment to open the file and look at it directly to see the difference.

Check for understanding: “What would happen if `run_log.txt` didn't exist yet at all, the very first time this script ran — would append mode fail?” (No — "a" mode, like "w" mode, creates the file if it doesn't already exist; the *only* difference between the two modes is what happens when the file *does* already exist: "w" erases it first, "a" doesn't. Worth stating this explicitly, since it resolves a reasonable “but what about the very first run” question a careful student might raise.)

7 Wrap-Up (last ~9 minutes)

Review the reflection questions out loud (answered as a comment at the top of the file):

1. *ETL in Python vs. Excel — what can each do that the other can't easily* — push for specifics: Python handles far larger files without manual scrolling, and the exact same script can be re-run instantly on updated data (Exercise 8's whole premise); Excel offers immediate visual inspection and ad-hoc exploration without writing any code at all — a strong answer names concrete tradeoffs, not just "Python is more powerful."
2. *A real append-mode log use case* — encourage something genuinely specific: an inventory system logging every stock change, a customer service tool logging every ticket resolution, a nightly batch job logging its own success/failure each night — the common thread worth naming explicitly is **a process that runs repeatedly over time, where each run's outcome is worth preserving, not just the most recent one.**

Review the submission checklist together:

- ☐ File is named `csv_practice.py`
- ☐ Contains Exercises 1–8, each clearly separated
- ☐ Reflection comment at the top of the file, answering both questions
- ☐ `week12/data/` contains `sales.csv`, `cleaned.csv`, and `run_log.txt` (with 3+ entries)
- ☐ Pushed to GitHub inside a `week12/` folder
- ☐ Repo URL submitted to Canvas

Preview Module 13: "Everything today used the built-in `csv` module, reading and writing one row at a time by hand. Next module introduces `pandas`, a library purpose-built for exactly this kind of tabular data work — today's whole ETL pipeline collapses into a handful of much shorter, more powerful lines."

8 Appendix A — Full Answer Key (`csv_practice.py`)

```
# csv_practice.py
# ISM2411 Module 12 Lab – Read a Sales CSV, Write a Cleaned Report
# Reflection:
# 1. [ETL in Python vs Excel – student's own words]
# 2. [Real append-mode log use case]

import csv
from datetime import datetime

# --- Exercise 2 ---
with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
```

```

    for row in reader:
        print(row)

# --- Exercise 3 ---
with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    print(header)
    for row in reader:
        print(row)

# --- Exercise 4 ---
total_revenue = 0
with open("week12/data/sales.csv") as f:
    reader = csv.reader(f)
    header = next(reader)
    for row in reader:
        product, price, quantity = row
        price = float(price)
        quantity = int(quantity)
        line_total = price * quantity
        total_revenue += line_total
        print(f"{product}: {quantity} units × ${price:.2f} = ${line_total:.2f}")
print(f"Total revenue: ${total_revenue:.2f}")

# --- Exercise 5 ---
rows_written = 0
rows_total = 0
with open("week12/data/sales.csv") as infile, \
    open("week12/data/cleaned.csv", "w", newline="") as outfile:
    reader = csv.reader(infile)
    writer = csv.writer(outfile)
    header = next(reader)
    writer.writerow(["Product", "Price", "Qty", "Total"])
    for row in reader:
        rows_total += 1
        product, price, quantity = row
        price = float(price)
        quantity = int(quantity)
        total = price * quantity
        if total > 100:

```

```

        writer.writerow([product, price, quantity, round(total, 2)])
        rows_written += 1
print(f"Wrote {rows_written} of {rows_total} rows")

# --- Exercise 6 ---
total_revenue = 0
with open("week12/data/sales.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        price = float(row["price"])
        quantity = int(row["quantity"])
        total_revenue += price * quantity
print(f"Total revenue (DictReader): ${total_revenue:.2f}")
# DictReader: access by column name, resilient to column reordering,
# more readable. csv.reader: simpler, slightly faster on huge files,
# fine when column order is guaranteed stable.

# --- Exercise 7 ---
summary = {}
with open("week12/data/sales.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        product = row["product"]
        total = float(row["price"]) * int(row["quantity"])
        summary[product] = summary.get(product, 0) + total
for product in sorted(summary):
    print(f"{product}: ${summary[product]:.2f}")

# --- Exercise 8 ---
def compute_total_revenue():
    total = 0
    with open("week12/data/sales.csv") as f:
        reader = csv.DictReader(f)
        for row in reader:
            total += float(row["price"]) * int(row["quantity"])
    return total

total_revenue = compute_total_revenue()
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open("week12/data/run_log.txt", "a") as f:
    f.write(f"{timestamp} | Total revenue: ${total_revenue:.2f}\n")

```

9 Appendix B — Sample sales.csv (verified)

Ten rows, deliberately including a mix of orders above and below the \$100 threshold used in Exercise 5, so the filter has something visible to exclude:

```
product,price,quantity
Widget A,9.99,50
Widget B,24.99,12
Gadget C,49.99,3
Gadget D,14.99,20
Gizmo E,99.99,2
Doohickey F,4.99,80
Widget A,9.99,5
Gadget C,49.99,1
Widget B,24.99,2
Gizmo E,99.99,1
```

All exercise outputs throughout this guide (including the exact dollar figures) were computed and verified against this exact file.

10 Appendix C — Extra Practice (only if the class finishes early)

Eight required exercises fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a second summary dimension. Using the same sales.csv, build a second summary dictionary tracking total *quantity* sold per product (not revenue) — same `.get(key, 0)` pattern, accumulating `int(row["quantity"])` instead of a computed dollar total. (Expected: Widget A: 55, Widget B: 14, Gadget C: 4, Gadget D: 20, Gizmo E: 3, Doohickey F: 80.)

Extra — a stricter filter, re-run. Modify Exercise 5's filter from `total > 100` to `total > 200`, write the result to a *new* file `cleaned_200.csv` (so the original `cleaned.csv` isn't overwritten), and print the new row count. (Expected: 4 of 10 rows qualify — Widget A/50, Widget B/12, Gadget D/20, and Doohickey F/80 — worth having students predict which four before running, based on Exercise 4's per-row totals already printed earlier in the file.)